A Reference Architecture for Data Quality in Federated Data Sharing

Dear participant,

To safeguard the quality of municipal services now and in the future, Common Ground was established. It is the information science vision in which the IT landscape is divided into components to exchange information based on federated data sharing. This disrupts the traditional landscape of monolithic systems operating in silos. However, little is known about how data quality can be guaranteed within federated data sharing. Although this was generally no issue within monolithic systems, this changed due to the distributed nature of federated data sharing. This knowledge gap is addressed within this study.

This research has realized a first version of a reference architecture as a solution to the data quality issue within federated data sharing. Based on requirements engineering, a set of constraint requirements, functional requirements, and quality requirements have been established and evaluated. Subsequently, a literature review was conducted to investigate which patterns could offer a solution, on which a trade-off analysis was performed based on the requirements. As a result, a reference architecture was realized, consisting of the architectural design (see figure 1) and the architectural design principles (see table 1).

You have been selected as an expert due to your knowledge of Common Ground to evaluate the proposed reference architecture. During the interview/ focus group, both artifacts will be evaluated, and your expertise will be requested regarding improvements such as the completeness, correctness, and applicability of the artifact.

Please feel free to familiarize yourself with these artifacts and make notes with comments or suggestions for improvement. Your participation is greatly appreciated.

I am always available for questions and comments. You can reach me at:

o.elkandoussi@students.uu.nl

1 Reference architecture for ensuring data quality

The reference architecture to evaluate consists of two parts. The first part is the architectural design, which includes the patterns, derived components, mechanisms, and the overview of the infrastructure. The second part is the architectural design principles, which form the foundation for realizing data quality within a federated data sharing architecture.

The architecture is based on eventual consistency to propagate data changes across the autonomous domains who are interested in the relevant data. The data sources maintain their autonomy and loose coupling with this architecture. The data changes are captured at their authoritative data source and propagated, which allows the data to become consistent eventually. However, for decisions (*'besluiten'*) data is retrieved directly from its source due the high importance that the data is accurate and up-to-date. This ensures decisions always being made based on the single source of truth which is the accurate and complete data within the federation.

2 The architectural design

The architectural design to achieve data quality is built using the following patterns: the transactional outbox, change data capture (CDC), and publish/subscribe (pub/sub). ArchiMate and UML sequence diagrams are used to document this artifact. ArchiMate is used to represent static components and their relationships, accompanied with a textual description. The UML sequence diagrams, documented in Appendix A, document the interactions between these components.

The chosen patterns introduced components that build on each other. The transactional outbox pattern is implemented through an outbox table. The outbox table is a database table in which data changes are recorded in the same atomic transaction as the domain table, with the goal to safeguard that the event from the publisher is reliably propagated to those interested in it [4, 7, 8]. The domain table holds the data of a specific domain. The CDC pattern introduces a component that enables log mining from the database's transaction log, in which data changes are detected and sent as events to the message broker [1, 6, 8]. The message broker, as implementation of the pub/sub pattern, is the component in the infrastructure through which publishers (who perform data changes) push events to and to which consumers (the interested data sources) receive events from in an asynchronous and loosely coupled way [1, 2, 9]. The event dispatcher, also known as the event service, is responsible for propagating events to the intended entities within the infrastructure [9].

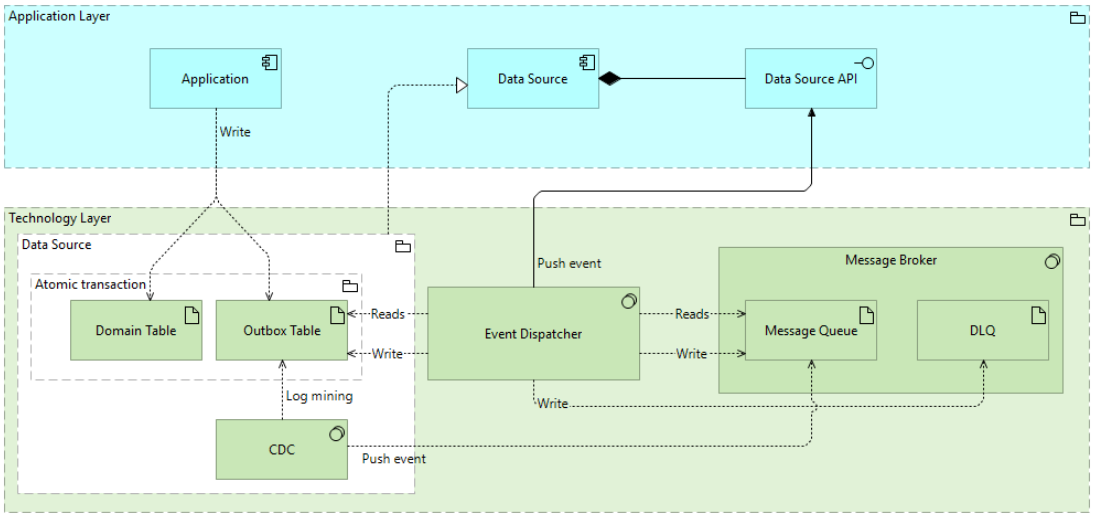

Figure 1: The Architectural Design for Ensuring Data Quality in Federated Data Sharing

The infrastructure ensuring data quality is under normal circumstances push-based. When the application changes a data object, an atomic write is done to the domain table and the outbox table. The CDC captures this data change from the outbox table and publishes an event to the message queue in the message broker. The message broker is known as Open Notificaties in Common Ground. The event dispatcher reads this event and writes to the outbox table that the event is published to the message broker. Subsequently, the event is being delivered to the subscribers through their data source API. The event is then removed from the message queue. The subscribers will retrieve the data themselves from the source in which the data change occurred. Figure 2 in Appendix A.1 represents the UML sequence diagram under normal circumstances.

Under the condition of failed components, components become pull-based to allow recovery of missed events. An outage of the message broker will not lead to missed events, as events are stored in the outbox table until successfully delivered to the message broker. Upon recovery, the event dispatcher pulls these missed events from the data sources' outbox tables, which are known by the value given to the attribute 'Published', and writes them to the message queue. This results in no events being lost when the message broker is down. The UML sequence diagram of this can be seen in figure 3, in Appendix A.2.

The events are written to the Dead-Letter Queue (DLQ) when the successful delivery of an event to a subscriber repeatedly fails, for example, due to the unavailability of that subscriber. By placing the events in the DLQ, the consumers become responsible, once restored, for retrieving their missed events from the message broker. This prevents the accumulation of messages to failed components in the message queue, which negatively impacts the continued delivery of events to available subscribers in the infrastructure. The UML sequence diagram of this can be seen in figure 3, in Appendix A.2.

3 The architectural design principles

To ensure correct implementation and execution of this architecture within the federated data sharing of Common Ground, a set of 6 principles is formulated and must be adhered to (see table 1). The framework template used for this was adapted from [3], which describes for each principle the aim, context, mechanism, and rationale. The *aim* specifies the purpose that the formulated architectural design principle intends to achieve. The *context* describes the relevant contextual factors and boundaries related to the principle. The *mechanism* describes the solution to safeguard the principle. The *rationale* explains the justification of the solution for the purpose of the principle [3].

Table 1: Architectural Design Principles for Data Quality in Federated Data Sharing

Principle	**Aim**	**Context**	**Mechanism**	**Rational**
Principle 1 - A decision shall only be taken based on the single source of truth.	To ensure that decisions are taken based on accurate data from its authoritative data source. The authoritative data source holds the single source of truth within the shared data layer.	Dutch law states the principle of due care (*'het zorgvuldigheidsbeginsel'*) in article 3:2 Awb [5], where it indirectly obligates to ensure information is accurate for decision making. Eventual consistency poses a risk to this.	Decisions are only taken based on data directly retrieved from its authoritative data source. From a CAP Theorem perspective, Consistency is chosen over Availability	Basing decisions on authoritative data sources ensures decisions are made on accurate data which minimizes legal risks and consequently possible harm to residents.
Principle 2 - Data in the federation shall only be modified at its source	To ensure data authority in federated data sharing, each specific data belongs only to one data source with a data owner.	With federated data sharing, data is distributed across the various autonomous domains. By allowing data modification in different data sources rather than the authoritative source, data quality becomes infeasible to ensure.	The data can only be modified at the authoritative data source. Data sources wanting to modify data cannot do that in their (external) data source if the data belongs to another source	By only allowing data modifications at the authoritative data source, this enables a single source of truth for data in the federation

Principle	Aim	Context	Mechanism	Rational
Principle 3 - A data source shall only be allowed to reference external data if it subscribes to it	To ensure that the data always remains consistent when data changes occur within the federated infrastructure.	As data changes continuously within the ecosystem, it is necessary that the dependent data sources (the consumers) are made aware the moment data changes. This allows them to act upon data changes.	A data source can only reference or retrieve data from an external source if it subscribes to that specific data in the message broker.	Subscribing to the authoritative data allows dependent entities to synchronize and act upon data changes automatically when they occur.
Principle 4 - A decision shall strictly record the time of the decision	To ensure that for each decision it is possible to identify which data it was based on and whether that data was accurate.	As in eventual consistency data over time converges to be consistent in the federation, it is necessary to be able to assess and reconstruct the data the decisions were based on.	For each decision, the decision timestamp and the data used are recorded.	Each data records the timestamps of the changes it had. Combining this with decisions that also are timestamped, we can verify and reconstruct the exact data the decision was based on.
Principle 5 - A revision process shall be in place for taken decisions	The aim is to review and revise a decision that has been made based on inconsistent data.	Eventual consistency has created a new playing field where we can never be 100% certain that all data is accurate and consistent. This must be taken into account when data was (temporarily) inconsistent prior to a decision.	The decision should be reviewed and revised when the data source receives events with related data changes that dates from before the decision being taken.	This principle fills the gap that principle 1 inherently cannot solve. It ensures that data that becomes eventually consistent after a decision is taken is still addressed.
Principle 6 - The events being processed shall be idempotent.	The purpose is to ensure that the same event does not lead to the same repeated change in the data source.	By using at-least-once delivery guarantee, it will happen that messages will be send and received more than once before it is successfully processed.	An idempotency key (or another unique identifier) needs to be added to each event to ensure that duplicate events can be detected.	The usage of such an unique identifier allows to differentiate between already events that are processed and not processed. This way ensuring at-least-once delivery does not lead to recurring data modifications as it is known to not process earlier events.

References

[1] Fernando, C.: Solution Architecture Patterns for Enterprise. Springer (2022)

[2] García, M.M.: Learn Microservices with Spring Boot: A Practical Approach to RESTful Services Using an Event-Driven Architecture, Cloud-Native Patterns, and Containerization. Springer (2020)

[3] Gregor, S., Chandra Kruse, L., Seidel, S.: Research perspectives: the anatomy of a design principle. Journal of the Association for Information Systems **21**(6), 2 (2020)

[4] de Oliveira Rosa, T., Daniel, J.F.L., Guerra, E.M., Goldman, A.: A method for architectural trade-off analysis based on patterns: Evaluating microservices structural attributes. In: Proceedings of the European Conference on Pattern Languages of Programs 2020. pp. 1–8 (2020)

[5] Overheid.nl: Algemene wet bestuursrecht (nd), `https://wetten.overheid.nl/BWBR0005537/2021-11-01/?g=2021-11-25&z=2021-11-25#Hoofdstuk3_Afdeling3.2_Artikel3:2`

[6] Perry, M.L.: The Art of Immutable Architecture: Theory and Practice of Data Management in Distributed Systems, Second Edition. Apress (2024)

[7] Reza, K., Rahman, N.: Explication and extension of saga and microservice patterns to enable resilient distributed transaction. In: Inventive Communication and Computational Technologies: Proceedings of ICICCT 2022, pp. 209–220. Springer (2022)

[8] Rocha, H.F.O.: Practical event-driven microservices architecture building sustainable and highly scalable event-driven microservices (2022)

[9] Tarkoma, S.: Publish/subscribe systems: design and principles. John Wiley & Sons (2012)

Appendix A - UML Sequence Diagrams Architectural Design

Appendix A.1 - UML Sequence Diagram Normal Condition

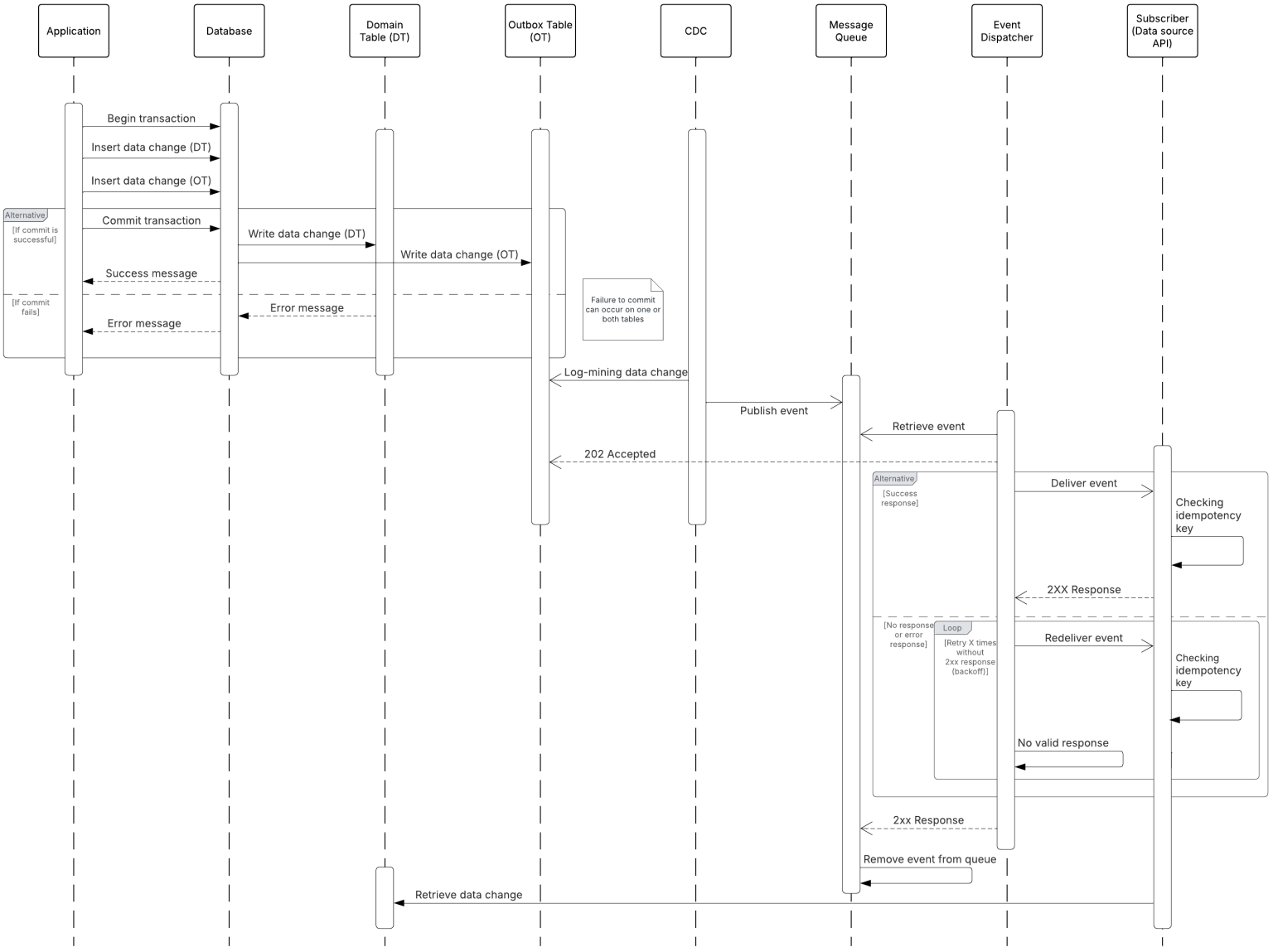

Figure 2: UML Sequence Diagram Architectural Design during Normal Condition

Appendix A.2 - UML Sequence Diagram Failed Components Condition

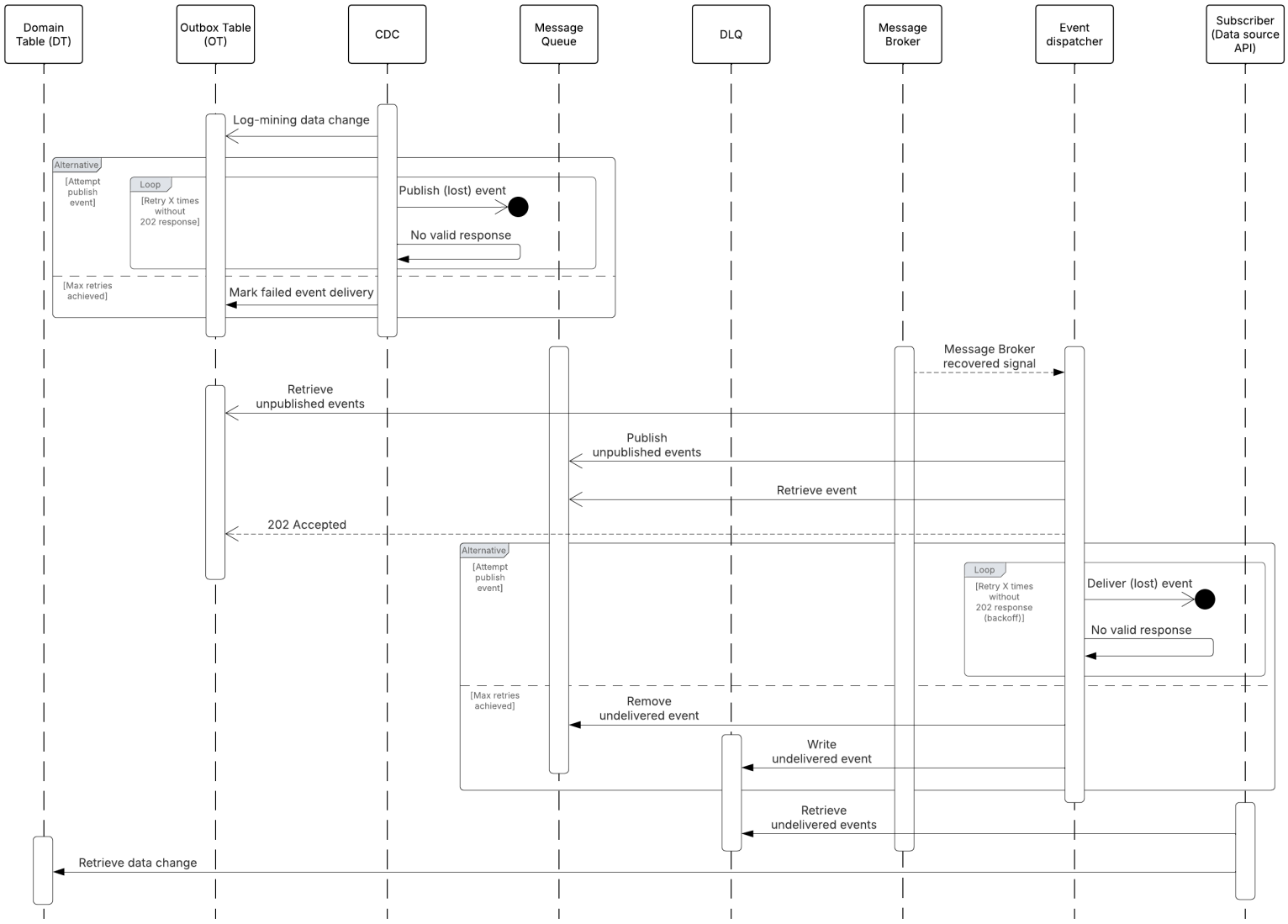

Figure 3: UML Sequence Diagram Architectural Design during Failure Condition